



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ-7 «Программное обеспечение ЭВМ и информационные технологии»

Отчет
по лабораторной работе № 6

Дисциплина: Функциональное и логическое программирование

Студент группы ИУ7-63Б

(Подпись, дата)

Паламарчук А.Н.

(Фамилия И.О.)

Преподаватель

(Подпись, дата)

Толпинская Н. Б.

(Фамилия И.О.)

Преподаватель

(Подпись, дата)

Строганов Ю.В.

(Фамилия И.О.)

2025 г.

Задание 1

Написать хвостовую рекурсивную функцию `my-reverse`, которая развернет верхний уровень своего списка-аргумента `lst`.

```
(defun my-reverse (lst &optional (acc nil))
  (cond ((null lst) acc)
        (t (my-reverse (cdr lst) (cons (car lst) acc))))
  ))

(print (my-reverse '(1 2 3 4))) ;→ (4 3 2 1)
(print (my-reverse '(a b c))) ;→ (c b a)
```

Задание 2

Написать функцию, которая возвращает первый элемент списка-аргумента, который сам является непустым списком.

```
(defun first-list (lst)
  (cond ((null lst) nil)
        ((and (listp (car lst)) (car lst)) (car lst))
        (t (first-list (cdr lst))))
  ))

(print (first-list '(a b (1 2) c))) ;→ (1 2)
(print (first-list '(x () (3) y))) ;→ (3)
```

Задание 3

Написать рекурсивную функцию, которая умножает на заданное число-аргумент все числа из заданного списка-аргумента, когда

- a) все элементы списка — числа,
- b) элементы списка — любые объекты.

```
(defun mul-num (lst n)
  (cond ((null lst) nil)
        (t (cons (* (car lst) n)
                    (mul-num (cdr lst) n))))
  ))
```

```

(print (mul-num '(1 2 3) 10)) ;→ (10 20 30)
(print (mul-num '(5 0 -2) 3)) ;→ (15 0 -6)

(defun mul-all (lst n)
  (cond ((null lst) nil)
        ((listp (car lst))
         (cons (mul-all (car lst) n)
               (mul-all (cdr lst) n)))
        ((numberp (car lst))
         (cons (* (car lst) n)
               (mul-all (cdr lst) n)))
        (t (cons (car lst)
                  (mul-all (cdr lst) n))))
  ))

(print (mul-all '(1 a 2 b) 2)) ;→ (2 a 4 b)
(print (mul-all '((1 2) a (3 b) 4) 2)) ;→ ((2 4) a (6 b) 8)
(print (mul-all '(1 (a (2)) (3 (4 b))) 10)) ;→ (10 (a (20)) (30 (40 b)))

```

Задание 4

Напишите функцию `select-between`, которая из списка-аргумента, содержащего только числа, выбирает только те, которые расположены между двумя указанными границами-аргументами и возвращает их в виде списка (упорядоченного по возрастанию списка чисел(+2балла)).

```

(defun insert (el lst)
  (cond ((null lst) (list el))
        ((<= el (car lst)) (cons el lst))
        (t (cons (car lst) (insert el (cdr lst)))))
  ))

(defun insertion-sort (lst)
  (cond ((null lst) nil)
        (t (insert (car lst) (insertion-sort (cdr lst)))))
  ))

```

```

(defun filter-between (lst a b)
  (let ((lower (min a b))
        (upper (max a b)))
    (cond ((null lst) nil)
          ((and (numberp (car lst))
                 (> (car lst) lower)
                 (< (car lst) upper))
           (cons (car lst)
                  (filter-between (cdr lst) a b)))
          (t (filter-between (cdr lst) a b))))))

(defun select-between (lst a b)
  (insertion-sort (filter-between lst a b)))

(print (select-between '(1 5 3 8 2 10) 2 6))      ; → (3 5)
(print (select-between '(10 20 30 15 25) 25 15)) ; → (20)
(print (select-between '(a 1 b 2 c 3) 3 0))      ; → (1 2)
(print (select-between '() 0 10))                ; → nil

```

Задание 5

Написать рекурсивную версию (с именем `rec-add`) вычисления суммы чисел заданного списка:

- a) одноуровневого смешанного;
- b) структурированного.

```

(defun rec-add (lst)
  (cond ((null lst) 0)
        ((numberp (car lst))
         (+ (car lst) (rec-add (cdr lst))))
        (t (rec-add (cdr lst))))))

(print (rec-add '(1 2 a 3))) ;→ 6

```

```

(print (rec-add '(10 x 20 y))) ;→ 30

(defun rec-add-str (lst)
  (cond ((null lst) 0)
        ((numberp (car lst))
         (+ (car lst) (rec-add-str (cdr lst))))
        ((listp (car lst))
         (+ (rec-add-str (car lst)) (rec-add-str (cdr lst))))
        (t (rec-add-str (cdr lst))))
  ))

(print (rec-add-str '(1 (2 a) (3 (4))))) ;→ 10
(print (rec-add-str '((5) x (y 6)))) ;→ 11

```

Задание 6

Написать рекурсивную версию с именем `recnth` функции `nth`.

```

(defun recnth (n lst)
  (cond ((or (null lst) (< n 0)) nil)
        ((= n 0) (car lst))
        (t (recnth (- n 1) (cdr lst))))
  ))

(print (recnth 2 '(a b c d))) ;→ c
(print (recnth 3 '(1 2 3 4))) ;→ 4
(print (recnth 4 '(1 2 3 4))) ;→ nil

```

Задание 7

Написать рекурсивную функцию `allodd`, которая возвращает `t` когда все элементы списка нечетные.

```

(defun allodd (lst)
  (cond ((null lst) t)
        ((not (numberp (car lst))) nil)
        ((evenp (car lst)) nil)
        (t (allodd (cdr lst))))
  ))

```

```
(print (allodd '(1 3 5))) ;→ t
(print (allodd '(1 2 3))) ;→ nil
```

Задание 8

Написать рекурсивную функцию, которая возвращает первое нечетное число из списка (структурированного), возможно создавая некоторые вспомогательные функции.

```
(defun first-odd (lst)
  (cond ((null lst) nil)
        ((numberp (car lst))
         (cond ((oddp (car lst)) (car lst))
               (t (first-odd (cdr lst)))))
        )
  ((listp (car lst))
   (or (first-odd (car lst)) (first-odd (cdr lst))))
  (t (first-odd (cdr lst)))
  ))

(print (first-odd '(2 4 (3) 6))) ;→ 3
(print (first-odd '((2 4) a (5 6)))) ;→ 5
```

Задание 9

Используя cons-дополняемую рекурсию с одним тестом завершения, написать функцию которая получает как аргумент список чисел, а возвращает список квадратов этих чисел в том же порядке.

```
(defun square-list (lst)
  (cond ((null lst) nil)
        (t (cons (* (car lst) (car lst))
                  (square-list (cdr lst))
                  )))
  ))

(print (square-list '(1 2 3))) ;→ (1 4 9)
(print (square-list '(0 -2 5))) ;→ (0 4 25)
```

Задание 10

Преобразовать структурированный список в одноуровневый.

```
(defun flatten-list (lst &optional (acc nil))
  (cond ((null lst) acc)
        ((atom lst) (cons lst acc))
        (t (flatten-list (car lst)
                          (flatten-list (cdr lst) acc)))
  ))

(print (flatten-list '(1 (2 (3))))) ;→ (1 2 3)
(print (flatten-list '((a) (b (c) d)))) ;→ (a b c d)
```